

RAPPORT DE PROJET

Phase 3 — Sécurité & Failles Volontaires

Projet : Secure API — Cybersécurité

Date : Mai 2026

1. Introduction

La Phase 3 est le coeur cybersécurité du projet. Elle consiste à implémenter un middleware d'authentification JWT, une route protégée, puis à introduire volontairement trois failles de sécurité majeures issues du Top 10 OWASP : la SQL Injection, le Cross-Site Scripting (XSS) et la mauvaise gestion des tokens JWT.

Chaque faille est d'abord démontrée en conditions réelles (attaque réussie), puis corrigée avec les bonnes pratiques. Cette approche démontre une compréhension approfondie des vulnérabilités — exactement ce qu'un pentester ou un développeur sécurité fait en entreprise.

1.1 Objectifs de la Phase 3

- Implémenter un middleware JWT pour protéger les routes sensibles
- Créer une route GET /users accessible uniquement aux utilisateurs authentifiés
- Démontrer et corriger une SQL Injection
- Démontrer et corriger une attaque XSS
- Démontrer et corriger une mauvaise gestion des tokens JWT
- Tester chaque faille avec Postman

2. Définitions des concepts clés

2.1 Concepts généraux de sécurité

OWASP (Open Web Application Security Project) : Organisation mondiale à but non lucratif qui publie le Top 10 des vulnérabilités web les plus critiques. C'est la référence mondiale en cybersécurité applicative. Chaque entreprise sérieuse s'appuie sur ce document pour sécuriser ses applications.

Pentest (Test d'intrusion) : Technique qui consiste à attaquer volontairement une application pour en découvrir les failles, avant qu'un vrai attaquant ne le fasse. C'est exactement ce qu'on simule dans cette phase : reproduire des attaques connues dans un environnement contrôlé.

Middleware : Fonction intermédiaire qui s'exécute entre la réception d'une requête HTTP et l'envoi de la réponse. Dans notre projet, le middleware JWT intercepte chaque requête sur les routes protégées pour vérifier la validité du token avant d'autoriser l'accès.

Endpoint vulnérable vs sécurisé : Pour chaque faille démontrée, on crée deux versions de l'endpoint : une version /vulnerable/ qui reproduit la faille, et une version /secure/ qui applique la correction. Cette approche permet de comparer les deux comportements en conditions réelles.

2.2 SQL Injection

SQL Injection : Faille qui permet à un attaquant d'injecter du code SQL malveillant dans un champ de saisie. Si l'application construit ses requêtes SQL en concaténant directement les inputs utilisateur, l'attaquant peut modifier la logique de la requête pour contourner l'authentification, lire ou supprimer des données.

Payload d'injection : La chaîne de caractères malveillante envoyée par l'attaquant. Dans notre démonstration : ' OR '1'='1. Cette chaîne ferme l'apostrophe de la requête originale et ajoute une condition toujours vraie, ce qui fait retourner tous les utilisateurs.

Requête préparée (Prepared Statement) : La correction contre les injections SQL. Au lieu de concaténer l'input dans la requête, on utilise un paramètre (?) que la bibliothèque remplace de manière sécurisée. L'input est toujours traité comme une donnée, jamais comme du code SQL.

Exemple concret de l'attaque :

```
Requête vulnérable : SELECT * FROM users WHERE email = '' OR '1'='1'
Décomposition      : WHERE email = '' OR '1'='1'
Résultat            : FAUX OR VRAI = VRAI → tous les utilisateurs retournés !
```

2.3 XSS (Cross-Site Scripting)

XSS (Cross-Site Scripting) : Faille qui permet à un attaquant d'injecter du code JavaScript malveillant dans une application web. Ce code sera ensuite exécuté dans le navigateur des autres utilisateurs. Le XSS est classé dans le Top 3 des failles web les plus courantes.

XSS Stocké (Stored XSS) : Type de XSS où le script malveillant est stocké en base de données. À chaque fois qu'un utilisateur consulte la page concernée, le script s'exécute. C'est la forme la plus dangereuse car elle affecte tous les visiteurs.

Sanitisation (Assainissement) : Processus de nettoyage des inputs utilisateur en remplaçant les caractères dangereux par leurs équivalents HTML inoffensifs. Par exemple, < devient < — le navigateur affiche le signe < mais ne l'interprète jamais comme une balise HTML.

Échappement HTML : Technique de correction contre le XSS. Les caractères spéciaux (<, >, ", ', /) sont remplacés par des entités HTML. Le contenu s'affiche correctement à l'écran mais ne peut jamais être exécuté comme du code.

Exemple concret de l'attaque :

```
Input malveillant : <script>alert('XSS - Vous etes hacke !')</script>
Version vulnera. : Bienvenue <script>alert('XSS')!</script> ! → script execute
```

```
Version securis. : Bienvenue
<script>alert(&#x27;XSS&#x27;)</script> ! → inoffensif
```

2.4 Mauvaise gestion des tokens JWT

Algorithme none (JWT) : Vulnérabilité JWT où le serveur accepte des tokens sans signature cryptographique. Un token JWT normal a 3 parties : HEADER.PAYLOAD.SIGNATURE. Avec l'algorithme none, la signature est absente. Si le serveur n'impose pas un algorithme spécifique, un attaquant peut forger un token avec n'importe quelles données (rôle admin par exemple).

jwt.decode() vs jwt.verify() : Deux fonctions fondamentalement différentes. `jwt.decode()` lit le contenu d'un token SANS vérifier sa signature — n'importe qui peut forger un token. `jwt.verify()` vérifie la signature cryptographique avec le secret du serveur — un token forgé sera immédiatement rejeté.

Forger un token : Créer un token JWT frauduleux avec des données inventées (userId, rôle, etc.) sans connaître le secret du serveur. Avec l'algorithme none, c'est possible car aucune signature n'est requise. C'est pourquoi il faut toujours imposer l'algorithme HS256.

Structure d'un token JWT forgé utilisé dans notre démonstration :

```
Header   : {alg: none, typ: JWT}
Payload  : {userId: 999, username: HACKER, role: admin}
Sign.    : (vide)
Token    :
eyJhbGciOiJIub251In0.eyJ1c2VySWQiOiJk5OSwidXNlcm5hbWUiOiJlIQUNLRVIiLCJyb2xlIjoiaY
WRtaW4ifQ.
```

3. Étapes réalisées

3.1 Middleware JWT

Le fichier `src/middleware/authMiddleware.ts` implémente le middleware d'authentification. Ce middleware s'intercale entre la réception de la requête et l'exécution de la route protégée.

Étape	Action	Détail technique
1	Récupérer le token	Extraction depuis le header Authorization : Bearer TOKEN
2	Vérifier la présence	Si token absent → 401 Accès refusé — Token manquant
3	Vérifier la signature	jwt.verify() avec le secret du serveur

4	Ajouter l'utilisateur	req.user = decoded → infos disponibles dans la route
5	Passer à la route	next() → la requête continue vers la logique métier

3.2 Route protégée GET /users

Le fichier `src/routes/userRoutes.ts` définit la route GET `/api/users`, protégée par le middleware JWT. Cette route retourne la liste des utilisateurs sans exposer les mots de passe hashés.

Test	Requête	Résultat
Sans token	GET <code>/api/users</code> (sans Authorization)	401 — Accès refusé Token manquant
Avec token valide	GET <code>/api/users</code> + Bearer TOKEN	200 — Liste des utilisateurs
Avec token expiré	GET <code>/api/users</code> + token expiré	401 — Token invalide ou expiré

3.3 Démonstration SQL Injection

Deux endpoints ont été créés dans `src/controllers/vulnerableController.ts` pour démontrer la faille et sa correction.

Version	Endpoint	Méthode	Comportement
Vulnérable	<code>/api/vulnerable/login</code>	POST	Concatène l'input directement dans le SQL
Sécurisée	<code>/api/vulnerable/secure-login</code>	POST	Utilise une requête préparée avec paramètre ?

Test	Input email	Version vulnérable	Version sécurisée
Email normal	<code>test@gmail.com</code>	200 — Connexion réussie	200 — Connexion réussie
Injection SQL	<code>' OR '1'='1</code>	200 — BYPASS !	401 — Utilisateur introuvable

3.4 Démonstration XSS

Deux endpoints ont été créés pour démontrer le stockage et la neutralisation d'un script malveillant.

Version	Endpoint	Comportement
Vulnérable	/api/vulnerable/register	Stocke et renvoie le username sans nettoyage
Sécurisée	/api/vulnerable/secure-register	Échappe les caractères dangereux avant réponse

Caractère	Avant nettoyage	Après nettoyage	Raison
<	<	<	Empêche d'ouvrir une balise HTML
>	>	>	Empêche de fermer une balise HTML
"	"	"	Empêche de sortir d'un attribut
'	'	'	Empêche d'autres injections
/	/	/	Empêche de fermer des balises

3.5 Démonstration mauvaise gestion JWT

Deux endpoints ont été créés pour démontrer l'acceptation d'un token forgé et sa correction.

Version	Endpoint	Fonction utilisée	Comportement
Vulnérable	/api/vulnerable/verify-token	jwt.decode()	Accepte tout token sans vérification
Sécurisée	/api/vulnerable/secure-verify-token	jwt.verify()	Vérifie signature + force HS256

Test	Token utilisé	Version vulnérable	Version sécurisée
Token légitime	Token généré au login	200 — Accès accordé	200 — Accès accordé
Token forgé admin	alg:none + role:admin	200 — BYPASS !	401 — Algorithme non autorisé

4. Récapitulatif des failles et corrections

Faillle	Catégorie OWASP	Attaque démontrée	Correction appliquée
SQL Injection	A03:2021 Injection	' OR '1'='1 dans le champ email	Requêtes préparées avec paramètres ?
XSS Stocké	A03:2021 Injection	<script>alert()</script> comme username	Échappement HTML des caractères dangereux
JWT Algorithm None	A02:2021 Broken Auth	Token forgé avec alg:none et role:admin	jwt.verify() + algorithmes: ['HS256']

5. Historique des commits GitHub

Commit	Description
feat: middleware JWT + route users protégée	Implémentation de l'authentification sur les routes
security: démonstration SQL Injection + correction	Faillle SQL et correction par requêtes préparées
security: démonstration XSS + correction	Faillle XSS et correction par échappement HTML
security: démonstration mauvaise gestion JWT + correction	Faillle JWT none et correction par jwt.verify()

6. Bilan

6.1 Ce qui a été réalisé

Fonctionnalité	Statut
Middleware JWT implémenté	Réalisé
Route GET /users protégée	Réalisé
Test sans token → 401 démontré	Réalisé
Test avec token → 200 démontré	Réalisé
SQL Injection démontrée et corrigée	Réalisé
XSS démontré et corrigé	Réalisé
JWT Algorithm None démontré et corrigé	Réalisé
Tous les tests Postman réussis	Réalisé
Commits GitHub documentés	Réalisé

6.2 Compétences démontrées

- Compréhension approfondie des failles OWASP Top 10
- Capacité à reproduire des attaques dans un environnement contrôlé
- Maîtrise des techniques de correction (requêtes préparées, échappement HTML, jwt.verify)
- Pensée offensive ET défensive — comprendre l'attaque pour mieux se défendre
- Documentation claire de chaque faille avec démonstration en live

6.3 Prochaines étapes — Phase 4

- Scanner les ports du serveur avec Nmap
- Intercepter et modifier les requêtes avec Burp Suite
- Scan automatique de vulnérabilités avec OWASP ZAP
- Rédaction du rapport final de sécurité